

A Full-Range Binary64 Exponential

LeanExe library PoC

September 2026

1 Task and verification status

`LeanExe.Lib.Transcendental.exp` computes e^x for binary64 inputs. The function accepts and returns IEEE 754 bit patterns in `UInt64` words. LeanExe compiles the Lean implementation and its clients to WebAssembly (WASM), which the examples execute in Wasmtime.

The implementation ports Arm’s 128-entry, degree-five exponential algorithm [1, 2]. It selects round-to-nearest, ties-to-even arithmetic and separate multiplication and addition. Arm reports a worst-case error of 0.511 ulp for this configuration without fused multiply-add. The port’s target is an error below one ulp.

The theorem `Project.ExpArm.Spec.exp_binary` proves that the recorded WASM bytes decode and validate to a module whose exponential terminates for every input word, preserves the complete store, and returns the result of an exact binary64 computation of the algorithm. The proof uses Talos’s WASM semantics with round-to-nearest, ties-to-even arithmetic and proves that instantiation initializes the lookup table. The theorem `exp_exact` applies to any store containing that table, supporting repeated calls.

For $|x| < 2^{-54}$, `exp_tiny_error` proves that the WASM returns one with absolute error below 2^{-53} against the real exponential. The full-range numerical error proof remains outstanding. Development tests compare WASM, the Lean port, Lean’s `Float.exp`, JavaScript’s `Math.exp`, and a high-precision decimal reference. The largest measured error for the port on 49,821 inputs is 0.504059 ulp.

2 Range reduction and evaluation

The reduction chooses an integer k near $128x/\log 2$ and writes

$$x = k \log(2)/128 + r, \quad e^x = 2^{k/128} e^r.$$

The implementation obtains k through a binary64 addition and subtraction of $1.5 \cdot 2^{52}$. Splitting $-\log(2)/128$ into two constants reduces the rounding error in r . The reduced argument has magnitude near or below $\log(2)/256$.

For $j = k \bmod 128$, a table supplies a leading value H_j and a correction t_j such that $2^{j/128} \approx H_j(1 + t_j)$. Its second word incorporates an exponent adjustment, allowing the scale to be formed by integer bit operations. The table has 128 pairs of 64-bit words, totaling 2 KiB.

The polynomial evaluates the correction

$$t_j + r + r^2(C_2 + rC_3) + r^4(C_4 + rC_5).$$

The coefficient words and evaluation order follow Arm’s source. Each arithmetic operation rounds separately. Normal reconstruction computes `scale + scale * tmp`.

For large positive inputs, reconstruction first reduces the scale’s exponent by 1009, evaluates the correction, and multiplies by 2^{1009} . For large negative inputs, it increases the scale’s exponent by 1022 and multiplies the corrected result by 2^{-1022} . When the corrected value is below one, a compensated sum rounds it to the precision required by the subnormal result before that multiplication. This step controls double rounding.

3 Input behavior and storage

Both signed zeros return one. Inputs with magnitude below 2^{-54} also return one. Positive infinity returns positive infinity. Negative infinity returns positive zero. All NaNs return the canonical quiet NaN with word `0x7FF8000000000000`. Finite results can overflow to positive infinity or underflow through subnormal values to positive zero.

The API specifies result values under round-to-nearest arithmetic. It has no error-number or floating-point exception-flag interface. Native Lean comparisons assume that the host uses round-to-nearest arithmetic and preserves subnormals.

The source uses a named `Array UInt64` literal. `LeanExe` places its length and words in a WASM data segment and emits bounds-checked loads for direct indexed reads. The allocation arena begins after the static data, and reset preserves the table. The execution theorem proves that an exponential call preserves all memory and globals. Array updates retain the compiler’s ordinary ownership and allocation behavior.

4 Executable implementation

The listing comes from the component’s `Basic.lean`. The table is in the adjacent `Table.lean`. The helper `ExpArm.rescale` implements the overflow and subnormal reconstruction described above.

```
def exp (x : UInt64) : UInt64 :=
  let magnitude := x &&& 0xFFFFFFFFFFFFFFFF
  if magnitude < 0x3C90000000000000 then 0x3FF0000000000000
  else if magnitude >= 0x4090000000000000 then
    if magnitude > 0x7FF0000000000000 then 0x7FF8000000000000
    else if x >>> 63 != 0 then 0
    else 0x7FF0000000000000
  else
    let z := mulBits 0x40671547652B82FE x
    let kBits := addBits z 0x4338000000000000
    let kd := subBits kBits 0x4338000000000000
    let r := addBits (addBits x (mulBits kd 0xBF762E42FEFA0000))
      (mulBits kd 0xBD0CF79ABC9E3B3A)
    let index := (2 * (kBits &&& 127)).toNat
    let tail := ExpArm.table[index]!
    let scale := ExpArm.table[index + 1]! + (kBits <<< 45)
    let r2 := mulBits r r
    let p := mulBits r2 (addBits 0x3FDFFFFFFFFFDBD (mulBits r 0x3FC5555555555543C))
    let q := mulBits (mulBits r2 r2)
      (addBits 0x3FA55555CF172B91 (mulBits r 0x3F81111167A4D017))
    let tmp := addBits (addBits (addBits tail r) p) q
    if magnitude >= 0x4080000000000000 then ExpArm.rescale tmp scale kBits
    else addBits scale (mulBits scale tmp)
```

5 Commands and a sigmoid client

The commands run from the repository root after following the setup guide.

```
tools/seminum exp 1
2.718281828459045
tools/seminum sigmoid 0
0.5
```

The decimal command accepts finite binary64 inputs. The hexadecimal command accepts every input word. The sigmoid client computes $\sigma(x) = 1/(1 + e^{-x})$ using $z = e^{-|x|}$: its result is $z/(1 + z)$ for negative inputs and $1/(1 + z)$ otherwise. This form preserves small positive results for large negative inputs without an overflowing intermediate exponential. Both clients import and call the library function.

6 Numerical tests

`node test/exp.js` runs 49,821 cases. It writes measurements to `build/seminum/exp-accuracy.json`. The corpus includes signed zeros, infinities, quiet and signaling NaNs, random binary64 words, inputs distributed across the finite output range, and neighboring words around reduction, overflow, and underflow boundaries. WASM and the Lean port agree bit for bit on every case. Separate tests cover the sigmoid client, command-line output, and allocation counts.

The reference uses Python’s standard-library `Decimal.exp`, which rounds to nearest at the requested decimal precision [4]. Conversion from the binary64 input is exact. Calculation starts at 100 significant digits and increases precision until the neighboring decimal bounds round to the same binary64 word. The error calculation uses those bounds. One ulp is the binary64 spacing in the exact result’s binade, with spacing 2^{-1074} for subnormal results.

Computation	Maximum error (ulp)	Differently rounded cases
LeanExe WASM	0.504059	49
Lean <code>Float.exp</code>	0.503354	36
JavaScript <code>Math.exp</code>	0.871814	4153

The last column counts differences from the reference’s rounded binary64 result, excluding NaN payloads. Lean 4.34.0-rc2 implements `Float.exp` through the external C function `exp` [3]. These measurements used Linux AArch64 and Node.js 24.13.0. Runtime-library comparisons depend on the installed implementations. Throughput and latency have yet to be measured.

References

- [1] Arm, *Double-precision exponential*. Source of the reduction, evaluation order, and reconstruction. The port retains Arm’s copyright notice and uses the MIT license.
- [2] Arm, *Shared exponential data*. Source of the table, split logarithm, coefficients, and reported error analysis.
- [3] Lean 4.34.0-rc2, *Floating-point API*. Documents the external C implementation of `Float.exp`.
- [4] Python, *Decimal arithmetic*. Documents exact conversion from binary floating point and correct rounding of the exponential at the selected decimal precision.